

1.4 — Implement Multi-Step Workflows with Enforcement and Handoff Patterns

Concept

Multi-step agentic workflows require more than just chaining prompts together. Each stage may produce output that the next stage depends on, and the correctness of that handoff determines whether the entire pipeline succeeds. There are two fundamentally different ways to ensure a workflow progresses correctly: **prompt-based guidance** and **programmatic enforcement**.

Prompt-based guidance relies on instructions in the system or user prompt to steer the model — telling it to "always return valid JSON" or "only proceed if the previous step succeeded." This is convenient but fragile: the model may deviate under long context pressure, unexpected input, or ambiguous situations. The prompt provides no hard guarantees. Programmatic enforcement, by contrast, validates outputs in code after each model response before passing them to the next stage. If the output fails validation, the workflow halts or retries — the model's good intentions are irrelevant.

The structured handoff pattern bridges these stages: each step produces a typed, validated payload that the next step receives as its input. Rather than passing raw text between steps, you define an interface for what each stage emits and parse the model's output against that schema. This creates a clear contract at every boundary. The general rule is: use deterministic code for anything that must be correct (schema validation, precondition checks, routing logic), and use model flexibility for anything that requires judgment (summarization, classification, generation). Mixing the two — trusting the model to enforce its own constraints — is the most common source of silent workflow failures.

Code Example

```
import Anthropic from "@anthropic-ai/sdk";
import { z } from "zod";

const client = new Anthropic();

// --- Stage schemas define the handoff contract ---

const ResearchOutputSchema = z.object({
  topic: z.string(),
  keyFindings: z.array(z.string()).min(1),
  confidence: z.enum(["low", "medium", "high"]),
});
```

```

const DraftOutputSchema = z.object({
  title: z.string(),
  body: z.string().min(100),
  wordCount: z.number().int().positive(),
});

type ResearchOutput = z.infer<typeof ResearchOutputSchema>;
type DraftOutput = z.infer<typeof DraftOutputSchema>;

// --- Programmatic enforcement via a typed step runner ---

async function runStep<T>(
  systemPrompt: string,
  userMessage: string,
  schema: z.ZodType<T>,
  maxRetries = 2
): Promise<T> {
  for (let attempt = 0; attempt <= maxRetries; attempt++) {
    const response = await client.messages.create({
      model: "claude-sonnet-4-5",
      max_tokens: 1024,
      system: systemPrompt,
      messages: [{ role: "user", content: userMessage }],
    });

    const text = response.content
      .filter((b) => b.type === "text")
      .map((b) => b.text)
      .join("");

    // Programmatic enforcement: parse and validate before proceeding
    try {
      const parsed = JSON.parse(text);
      return schema.parse(parsed); // throws ZodError if schema not satisfied
    } catch (err) {
      if (attempt === maxRetries) {
        throw new Error(
          `Step failed schema validation after ${maxRetries + 1} attempts: ${err}`
        );
      }
      // Could inject the validation error back into the next attempt here
    }
  }
  throw new Error("Unreachable");
}

```

```
// --- Structured handoff: each stage receives the previous stage's validated output ---

async function researchStage(topic: string): Promise<ResearchOutput> {
  return runStep(
    'You are a research assistant. Respond ONLY with a JSON object matching: { "topic": string, '
    `Research this topic and return structured findings: ${topic}`,
    ResearchOutputSchema
  );
}

async function draftStage(research: ResearchOutput): Promise<DraftOutput> {
  // The handoff: structured context from stage 1 becomes the input for stage 2
  const handoffContext = JSON.stringify(research, null, 2);

  return runStep(
    'You are a technical writer. Respond ONLY with a JSON object matching: { "title": string, '
    `Using this research, write a short article:\n\n${handoffContext}`,
    DraftOutputSchema
  );
}

// --- Orchestrator: deterministic routing, not prompt-guided ---

async function runWorkflow(topic: string): Promise<DraftOutput> {
  const research = await researchStage(topic);

  // Deterministic gate: model judgment not trusted for routing decisions
  if (research.confidence === "low") {
    throw new Error(
      `Research confidence too low for topic "${topic}". Aborting draft stage.`
    );
  }

  return draftStage(research);
}
```

Common Mistakes

1. **Trusting the model to self-validate** — Adding "ensure your output is valid JSON" to a prompt is not enforcement. Parse and validate in code; if the schema check fails, the step failed regardless of what the model intended.
2. **Passing raw text between stages** — Handing the raw string output of one step directly into the next prompt makes the pipeline brittle. A single unexpected phrase or formatting quirk in stage one breaks stage two. Always parse into a typed structure at each boundary.

3. **Using model output for routing decisions** — Letting the model decide whether to proceed to the next stage (e.g., "if you think the data is ready, say PROCEED") mixes concerns incorrectly. Routing logic belongs in deterministic code; use schema-validated fields like `confidence` or `status` to drive control flow.

References

- [Anthropic: Build with Claude — Workflows and orchestration](#)
- [Anthropic: Tool use overview](#)
- [Zod schema validation](#)